05 — Issuer verification

What it detects: a document issued by someone who is not a known, active vendor/school/issuer — including fabricated or look-alike issuer names.

Code: `services/issuer_service.py` (`match_issuer`), `models/issuer_registry.py`, run by `tasks/document_checks.py` (`run_document_checks`), which stores a `document_checks` row with `check_type = issuer_verification`. Registry administration: `api/settings.py` `/settings/issuers` and the *Settings › Issuer Registry* page (a company's Reviewer L2s; platform admins on a company's behalf).

The registry is per company. Each company maintains its own list of known issuers; a document is only ever matched against **its own company's** registry (`issuer_registry.company_id`, enforced by the tenant-bound session and Row-Level Security).

Algorithm

`match_issuer(db, company_id, issuer_name, threshold=None, llm_service=None) -> IssuerMatchResult`:

1. Empty/blank issuer name → no match.
2. Load every **active** `issuer_registry` row **of the document's company** (deactivated issuers no longer count as known). If none → no match.
3. Build candidate strings: each row contributes its `name` **and**, if present, its `name_arabic`, so an Arabic-script extracted name can match a row whose primary name is English.
4. **Fuzzy pass:** `rapidfuzz.process.extractOne(issuer_name, candidates, scorer=fuzz.WRatio, processor=utils.default_process)` — case-, punctuation- and whitespace-insensitive. Match if the best score ≥ threshold (`ISSUER_FUZZY_MATCH_THRESHOLD`, default **85.0**).
5. **LLM fallback** (only if the fuzzy pass fails and an `LLMService` was passed — the production task passes one): `judge_entity_match(name, candidates)` asks Azure OpenAI whether the extracted name plausibly refers to the same real-world entity as any candidate — this covers transliteration and cross-script cases rapidfuzz cannot score. The model must return one of the candidates **verbatim** or `null`. An LLM failure or missing configuration is swallowed and reported as the fuzzy-only result. The call runs on `vision_queue` through the global Azure OpenAI rate limiter (`max_tokens` 512).

```
@dataclass IssuerMatchResult:
    matched: bool
    best_match_name: str | None
    best_match_score: float | None       # rapidfuzz score, rounded to 2 dp
    threshold: float
    matched_via: Literal["fuzzy", "llm"] | None
```

Output shape (real example)

`document_checks.result` for an issuer that did not match:

```
{
  "result": "flag",                                // pass | flag
  "details": {
    "issuer_name": "بنك أبوظبي الأول | First Abu Dhabi Bank",
    "threshold": 85.0,
    "match_score": 27.94,
    "matched_via": null,                           // "fuzzy" | "llm" | null
    "matched_registry_name": "Al Nukhba Technical Systems Est."   // the closest, even when not a match
  }
}
```

`details` is a **flat dict**, not a findings list. `matched_registry_name` names the closest registry entry even when the check flags, so a reviewer can see what it was nearest to.

Registry data

`issuer_registry` columns: `company_id`, `name`, `name_arabic`, `tax_id`, `type` (enum), `is_active`. Registry actions (create / update / deactivate / reactivate) each write an audit event in the company's log; **rows are never deleted**. A **new company's registry starts empty** — its Reviewer L2s fill it in. The migrations seeded a handful of **fake** test issuers into the "Default Company" created when the single-tenant database was migrated. Names must be unique case-insensitively among a company's active issuers (409 otherwise).

Known limitations

- **A new company's registry is empty, and the Default Company's seeded one is fake.** Until a company's registry holds its real vendors/schools, almost every one of its documents will flag.
- **Names only:** `tax_id` is stored and editable but not compared to the extracted document.
- The LLM fallback is a judgment call that can be wrong in either direction, and it can only choose among registry strings — it cannot verify that the issuer exists in the real world.
- No logo/letterhead matching.
- A single extracted issuer string is checked; multi-party documents (issuer + bank) may extract the "wrong" party as the issuer.

Risk rules fed

`rule_id`	Condition	Weight	Severity
`issuer.not_in_registry`	`issuer_verification` result = `flag`	15	medium